AI:

Prompt:

Write C++ code that does the following:

Suppose we have a Sudoku puzzle of which k of the 81 cells
are unfilled. We identify an empty cell and deduce a list of candidates for
what might go into that cell. If there are no such values (all digits are
accounted for elsewhere in the same row, column, or grid), then we must have
made a mistake in filling in the k cells and return a no-solution-found result.
Otherwise, we try putting the first candidate value into the empty cell, and see
if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we
get a no-solution-found to the first candidate, then we try the next, and so on,
until we either find a solution (k == 81) or determine that no solution exists
(all candidates report no solution, which again implies we took a wrong path
somewhere earlier).

You are given five files (puzzle1.txt through puzzle5.txt), each containing a
Sudoku puzzle. Digits already filled in have been placed, and an underscore
character (_) is used to indicate a blank square.

Write a recursive object-oriented program to find the solution to the five
puzzle files.
The output for each solution, should include:

- The puzzle file name (e.g., puzzle1.txt)
- The puzzle from the file printed out.
- The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.

The solution may not be unique; if there is more than one solution to the
puzzle, your program should print them all out.

Here is an example of puzzle1.txt:

```
5 _ _ _ _ _ 1 7 _
1 _ 6 5 _ 9 _ 4 _
4 7 2 1 _ 6 _ _ _
9 _ _ _ _ _ 5 _ _
_ 1 8 _ 9 5 4 _ _
6 _ _ 4 _ 2 3 8 9
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4
```

Here is the solution for puzzle1:

```
5 3 9 8 2 4 1 7 6
```

1 8 6 5 7 9 2 4 3
4 7 2 1 3 6 8 9 5
9 2 4 3 8 7 5 6 1
3 1 8 6 9 5 4 2 7
6 5 7 4 1 2 3 8 9
7 4 5 2 6 1 9 3 8
8 9 1 7 4 3 6 5 2
2 6 3 9 5 8 7 1 4

Solutions found: 1

Remember to include all of the required output in the output.

Gemini Implementation:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
    std::vector<std::vector<char>> grid;
    std::string filename;
    int solutionsCount;

    // Helper to print a 9x9 grid in the specified format
    void printGrid(const std::vector<std::vector<char>>& targetGrid) const {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                std::cout << targetGrid[r][c] << (c == 8 ? "" : " ");
            }
            std::cout << "\n";
        }
    }

    // Gathers valid candidate digits for an empty cell
    std::vector<char> getCandidates(int row, int col) const {
        std::vector<bool> available(10, true);

        // Check row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] != '_') available[grid[row][i] - '0'] = false;
            if (grid[i][col] != '_') available[grid[i][col] - '0'] = false;
        }

        // Check 3x3 sub-grid constraints
        int startRow = (row / 3) * 3;
```

```
    int startCol = (col / 3) * 3;
    for (int r = startRow; r < startRow + 3; ++r) {
        for (int c = startCol; c < startCol + 3; ++c) {
            if (grid[r][c] != '_') {
                available[grid[r][c] - '0'] = false;
            }
        }
    }

    // Compile all allowed digits
    std::vector<char> candidates;
    for (int i = 1; i <= 9; ++i) {
        if (available[i]) {
            candidates.push_back(static_cast<char>('0' + i));
        }
    }
    return candidates;
}

// Finds the next empty cell ('_'). Returns false if the grid is entirely full.
bool findEmptyCell(int& row, int& col) const {
    for (row = 0; row < 9; ++row) {
        for (col = 0; col < 9; ++col) {
            if (grid[row][col] == '_') {
                return true;
            }
        }
    }
    return false;
}

// Recursive solver function
void solveRecursive() {
    int row, col;

    // Base case: If no empty cells remain, a valid configuration has been completed
    if (!findEmptyCell(row, col)) {
        solutionsCount++;
        std::cout << "\nSolution " << solutionsCount << ":\n";
        printGrid(grid);
        return;
    }

    // Get candidates for the current empty cell
    std::vector<char> candidates = getCandidates(row, col);

    // Branching / Backtracking step
    for (char candidate : candidates) {
```

```
            grid[row][col] = candidate; // Tentatively place candidate
            solveRecursive();           // Recurse to solve the remaining empty cells
            grid[row][col] = '_';       // Reset cell upon return (backtrack)
        }
    }

public:
    SudokuSolver(const std::string& file) : filename(file), solutionsCount(0), grid(9,
std::vector<char>(9, '_')) {}

    bool loadPuzzle() {
        std::ifstream inFile(filename);
        if (!inFile) {
            std::cerr << "Error: Could not open file " << filename << "\n";
            return false;
        }

        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (!(inFile >> grid[r][c])) {
                    std::cerr << "Error: Invalid grid size or formatting in " << filename << "\n";
                    return false;
                }
            }
        }
        return true;
    }

    void process() {
        std::cout << "========================================\n";
        std::cout << "Puzzle File Name: " << filename << "\n";
        std::cout << "========================================\n\n";
        std::cout << "Original Puzzle:\n";
        printGrid(grid);

        solutionsCount = 0;
        solveRecursive();

        if (solutionsCount == 0) {
            std::cout << "\nNo solution found\n";
        } else {
            std::cout << "\nSolutions found: " << solutionsCount << "\n";
        }
        std::cout << "\n";
    }
};

int main() {
```

```
    // List of puzzle files to read and process sequentiallly
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const std::string& file : puzzleFiles) {
        SudokuSolver solver(file);
        if (solver.loadPuzzle()) {
            solver.process();
        }
    }

    return 0;
}
```

ChatGPT Implementation:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <iomanip>

using namespace std;

class Sudoku {
private:
    static const int SIZE = 9;
    int board[SIZE][SIZE];
    int original[SIZE][SIZE];
    int solutionCount;

    // Check whether a digit can be placed in a cell.
    bool isValid(int row, int col, int digit) const {
        // Check the row and column.
        for (int i = 0; i < SIZE; i++) {
            if (board[row][i] == digit)
                return false;

            if (board[i][col] == digit)
                return false;
        }

        // Find the top-left corner of the 3x3 grid.
```

```
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    // Check the 3x3 grid.
    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if (board[r][c] == digit)
                return false;
        }
    }

    return true;
}

// Recursively find and print every solution.
void solveRecursive() {
    int bestRow = -1;
    int bestCol = -1;
    int fewestCandidates = 10;

    // Find an empty cell with the fewest valid candidates.
    // This optimization can significantly reduce the search time.
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (board[row][col] != 0)
                continue;

            int candidateCount = 0;

            for (int digit = 1; digit <= 9; digit++) {
                if (isValid(row, col, digit))
                    candidateCount++;
            }

            // An empty cell with no candidates means this path
            // cannot lead to a solution.
            if (candidateCount == 0)
                return;

            if (candidateCount < fewestCandidates) {
                fewestCandidates = candidateCount;
                bestRow = row;
                bestCol = col;
            }
        }
    }

    // No empty cells remain, so a complete solution was found.
```

```
        if (bestRow == -1) {
            solutionCount++;

            cout << "\nSolution " << solutionCount << ":\n";
            printBoard(board);
            return;
        }

        // Try every valid candidate for the selected empty cell.
        for (int digit = 1; digit <= 9; digit++) {
            if (isValid(bestRow, bestCol, digit)) {
                board[bestRow][bestCol] = digit;

                // Recursively solve the remaining empty cells.
                solveRecursive();

                // Backtrack to try the next candidate.
                board[bestRow][bestCol] = 0;
            }
        }
    }

public:
    Sudoku() : solutionCount(0) {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                board[row][col] = 0;
                original[row][col] = 0;
            }
        }
    }

    // Read a Sudoku puzzle from a file.
    bool loadFromFile(const string& filename) {
        ifstream inputFile(filename);

        if (!inputFile) {
            cerr << "Error: Could not open " << filename << '\n';
            return false;
        }

        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                string value;

                if (!(inputFile >> value)) {
                    cerr << "Error: Invalid puzzle format in "
                        << filename << '\n';
```

```
                return false;
            }

            if (value == "_") {
                board[row][col] = 0;
            } else {
                try {
                    size_t pos = 0;
                    int digit = stoi(value, &pos);

                    if (pos != value.size() ||
                        digit < 1 || digit > 9) {
                        cerr << "Error: Invalid cell value in "
                             << filename << '\n';
                        return false;
                    }

                    board[row][col] = digit;
                } catch (...) {
                    cerr << "Error: Invalid cell value in "
                         << filename << '\n';
                    return false;
                }
            }
        }
    }

    // Save the original puzzle for display.
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            original[row][col] = board[row][col];
        }
    }

    return true;
}

// Print a Sudoku board, displaying zeroes as underscores.
void printBoard(const int grid[SIZE][SIZE]) const {
    for (int row = 0; row < SIZE; row++) {
        for (int col = 0; col < SIZE; col++) {
            if (grid[row][col] == 0)
                cout << "_";
            else
                cout << grid[row][col];

            if (col < SIZE - 1)
                cout << ' ';
```

```
            }
            cout << '\n';
        }
    }

    // Print the original puzzle and find all solutions.
    void solve(const string& filename) {
        cout << "\n========================================\n";
        cout << "Puzzle file: " << filename << '\n';
        cout << "========================================\n";

        cout << "\nOriginal puzzle:\n";
        printBoard(original);

        solutionCount = 0;

        solveRecursive();

        if (solutionCount == 0) {
            cout << "\nNo solution found\n";
        }

        cout << "\nSolutions found: " << solutionCount << "\n";
    }
};

int main() {
    vector<string> filenames = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const string& filename : filenames) {
        Sudoku puzzle;

        if (puzzle.loadFromFile(filename)) {
            puzzle.solve(filename);
        }
    }

    return 0;
}
```

Analysis:

Gemini (Accessed through Google Search query):

Correctness:
Gemini correctly implements recursive backtracking to solve the Sudoku puzzles. The algorithm goes through and finds every possible solution to the puzzle, printing them as it goes when it is done. It also correctly prints that there is no solution found if the puzzle cannot be solved. As required, the design is object oriented. Some fallbacks of the program are that it does not check if a puzzle has an invalid start. If the puzzle is fully filled in but not a correct solution, it does not detect it, because the base case on the recursive solve only checks if the grid is full. The algorithm also does not immediately end if a partially solved puzzle that is invalid is given. The algorithm also does not check for bad input, only digits 1-9 and _.

Time Complexity:
The time complexity for solving a puzzle in the worst case is O(9^k), where k is the number of empty cells. The findEmptyCell() function could be faster, as the function just gets the next cell in row-major order rather. A faster approach would be to find the next cell with the fewest possible values to prevent unnecessary branching and speed up the recursion process.

Space Complexity:
The space complexity of the algorithm is solid, and is O(81) to store the board since there are 81 cells. The program does not take up unneeded space as it prints the solution as soon as it is found rather than storing it.

Maintainability:
Gemini has a concise algorithm for finding the puzzle solutions, and has good readability for the recursive calls. The helper functions have specific tasks that make them easy to use and follow. The code has comments describing the general use of each coding block, but isn't too specific. Interestingly enough, Gemini made a spelling error in one of its comments, spelling "sequentially", as "sequentiallly", with an extra l.

ChatGPT:

Correctness:
ChatGPT correctly implements recursive backtracking to solve the Sudoku puzzles. The algorithm goes through and finds every possible solution to the puzzle, and just like Gemini, prints the solutions once they are found. It also correctly prints that there is no solution found if the puzzle cannot be solved, and the design is object oriented as required. Compared to Gemini, ChatGPT does account for input other than digits 1-9 and _, but still fails in the same place of not detecting if a puzzle is given in an invalid state. If a board is invalid, the algorithm does not immediately detect it. And while the program does check for other input, it does not account for extra input after the puzzle.

Time Complexity:
ChatGPT's time complexity is similar to Gemini's being O(9^k) as well, where k is the number of empty cells. ChatGPT improves on the time complexity compared to Gemini however, as it does find the cell with the least amount of possible values rather than just preceding onwards to the next cell.

Space Complexity:
Space complexity is also similar to Gemini's, being O(81) for the 81 cells that the board stores. The program also does not store the solutions to the puzzles, printing them out immediately once they have been found instead, so it does not take extra space saving them.

Maintainability:
In terms of maintainability, ChatGPT has a good object oriented approach that has clear function purposes, and good error handling. ChatGPT does include the iomanip library in the code, despite not using it. Comments are a little better than Gemini's, as it has comments for major blocks and explains them slightly better.

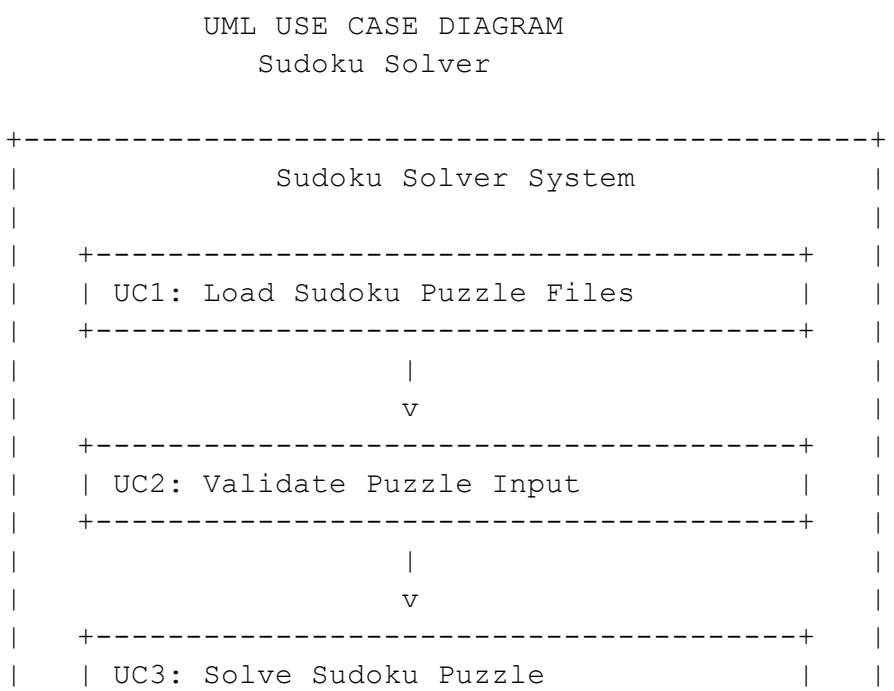
Winner:

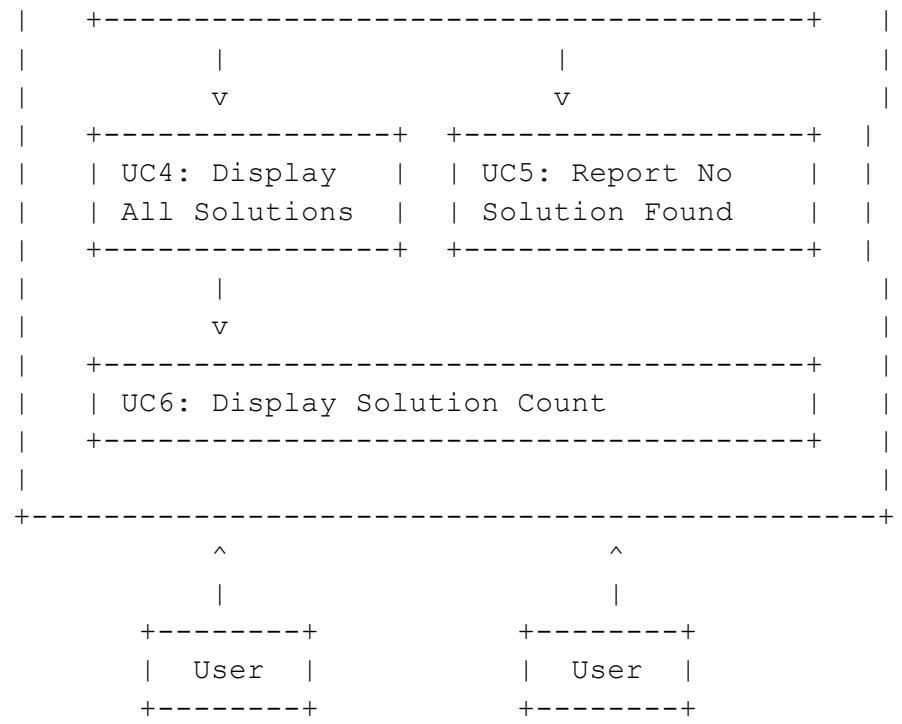
I decided to use ChatGPT's code over Gemini's. ChatGPT has a faster algorithm that is less prone to bad input, while still solving the puzzles in the same way. With improvements to correctness and time complexity with barely any difference in space complexity, ChatGPT was a clear winner here. In addition, Gemini made a spelling mistake, which is, of course, unforgivable, and immediately invalidates all of its code.

Improvements:

I improved ChatGPT's code by removing the iomanip library that went unused. I added a validateBoard() function that would ensure the given puzzle was an actual valid puzzle. I added some extra input handling to ensure the board is correct. I added a comment to every line of code to improve maintainability, and I made sure there were no spelling mistakes.

UML Use Case Diagram:





Use case name: Solve Sudoku Puzzles

Primary actor: User

Goal: Find and display all valid solutions to Sudoku puzzles stored in puzzle1.txt through puzzle5.txt.

Trigger: The user runs the Sudoku Solver program.

Preconditions:

The program has been compiled successfully.

The puzzle files are available in the expected directory.

Each puzzle file contains 81 cell values, with underscores representing empty cells.

Postconditions:

The original puzzle is displayed.

Every valid solution is displayed, or a no-solution message is shown.

The total number of solutions is reported.

Use Case Description:
The user runs the Sudoku Solver program, which reads each puzzle file, validates the input and pre-filled digits, and attempts to solve the puzzle using recursive backtracking. The

system prints all valid solutions and the total solution count. If the puzzle is invalid or has no solution, the system reports the appropriate error or no-solution result.

Step-by-step flow of events:

1. The user starts the program.
2. The system identifies the five puzzle filenames.
3. The system opens the first puzzle file.
4. The system reads the 81 cell values.
5. The system checks the input format and validates the pre-filled digits.
6. The system prints the original puzzle.
7. The solver identifies an empty cell with the fewest valid candidates.
8. The solver tries a candidate and recursively searches for a solution.
9. If a candidate leads to a dead end, the solver backtracks and tries another candidate.
10. When a complete valid board is found, the system prints the solution and increments the solution count.
11. The system continues searching until all candidate paths have been explored.
12. The system prints the total number of solutions, or No solution found if no solution exists.
13. The system repeats the process for the remaining puzzle files.
14. The program terminates after processing all five files.

Detailed description:

Use case ID:

UC-01

Use case name:

Solve Sudoku Puzzles

Primary actor:

User

Supporting system:

File system

Description:

Reads, validates, solves, and displays Sudoku puzzles.

Trigger:

The user executes the program.

Preconditions:

The program is compiled and the puzzle files are accessible.

Main flow:

Load puzzle, validate input, display original board, recursively solve, print all solutions, and report the solution count.

Alternative flow A:

If a puzzle file cannot be opened, the system reports an error and continues with the next file.

Alternative flow B:

If the puzzle format is invalid or the initial clues conflict, the system reports an error and skips that puzzle.

Alternative flow C:

If a valid puzzle has no solution, the system displays No solution found and reports zero solutions.

Postconditions:

The results for each successfully loaded and validated puzzle are displayed.

Main success scenario:
The user starts the program. The system reads and validates a Sudoku puzzle, prints the original board, and uses recursive backtracking to explore candidate values. Whenever a complete valid solution is found, it is displayed. The solver continues exploring until all possible solutions have been found. Finally, the system displays the number of solutions and proceeds to the next puzzle file.

Alternative scenarios:

File unavailable: The system prints an error message and moves on to the next file.

Invalid input: The system rejects the malformed puzzle.

Conflicting initial clues: The system rejects the puzzle before starting the recursive search.

No valid solution: The system reports that no solution was found.

Multiple solutions: The system displays every solution and reports the total count.